Security Risks of Vibe Coding: Vulnerabilities Analysis of AI-generated Implementations in Banking Applications

Student ID: 2023320129, Name: Intan Nurnazihah
Advisor: Heejo Lee

2026.6.2

Abstract

This study analyzes security risks in AI-generated applications, focusing on banking systems that handle sensitive financial data and user credentials. A prototype banking application is generated using ChatGPT with security-critical features such as user authentication, secure transaction processing, and cryptographic key management using the Advanced Encryption Standard. The generated code is evaluated using static security analysis tools like Semgrep to identify potential vulnerabilities and insecure cryptographic implementations. The results aim to highlight security risks associated with AI-generated applications in security-sensitive environments.

1 Introduction

The rapid advancement of generative artificial intelligence has significantly influenced modern software development practices. In recent years, developers have increasingly relied on AI tools such as ChatGPT, Gemini, and Claude to generate code through natural language prompts. This emerging paradigm, often referred to as *vibe coding* which enables rapid prototyping and reduces the need for manual implementation.

One of the main motivations behind vibe coding is its ability to significantly improve development speed and efficiency. Prior studies report that developers are able to produce functional software quicker than traditional development time. In most cases, it is reported that projects are able to be completed within hours instead of weeks. In addition to speed, vibe coding also improves accessibility and empowers individuals without formal software development backgrounds to build applications. They just simply need to describe their requirements to the AI tools using natural language. This broadens the field of software development by not limiting it to those with formal backgrounds only. Furthermore, vibe coding supports rapid prototyping. It allows developers to quickly assemble minimum viable products (MVPs) or demonstration systems to evaluate feasibility, explore new ideas and keep up with the market [1].

However, the high dependence on AI raises concerns regarding the reliability and security of automatically generated codes. Developers tend to prioritize rapid completion of tasks over careful verification of the generated code, which can lead to insecure coding practices being incorporated into real systems. One of the key contributing factors is the comprehension gap, where developers deploy AI-generated code without fully understanding its underlying logic. In such cases, "shadow code", where AI outputs are directly reused with minimal modification, increases the likelihood of hidden vulnerabilities. Prior studies have shown that AI-generated code can contain significantly more defects and security issues compared to human-written implementations. This issue is particularly concerning in security-sensitive applications where vulnerabilities introduced during development may compromise the integrity and confidentiality of the entire system.

As AI-assisted development becomes increasingly

common, understanding the risks associated with AI-generated code is essential. This study evaluates the reliability of generative AI in producing secure application components by analyzing the security quality of AI-generated implementations and identifying potential risks in modern software engineering.

2 Overview

This research examines the possible security challenges posed by AI-generated code within banking applications. As large language models become increasingly integrated into software development, they enable rapid code generation through natural language prompts. While this accelerates productivity, it also raises critical concerns about the security when being dependent of automatically produced code. This is particularly relevant in systems managing sensitive financial information.

The study centers on a prototype banking system that implements essential features such as user authentication, transaction handling and cryptographic operations. Its goal is to determine whether AI-generated code can securely deliver these core functions without embedding hidden vulnerabilities both at syntactic and logic levels.

These features were selected because they represent core components commonly found in financial applications and require careful security implementation. Authentication controls access to user accounts, transaction handling ensures that financial operations are processed correctly and cryptographic operations protect sensitive data. Therefore, evaluating these components provides a focused way to examine whether AI-generated code can meet the security expectations of a banking system.

Unlike approaches that focus mainly on functional accuracy, this investigation prioritizes uncovering potential security flaws that may emerge from AI-assisted development. By combining static and dynamic analysis, the evaluation covers both code-level weaknesses and runtime risks thus offering a thorough security assessment.

This research does not aim to reject the use of AI-generated code. Instead, it treats AI-generated code as an initial implementation that must be reviewed and tested before deployment. This perspective is important because AI-assisted development can be useful but its output should not be assumed secure without a proper evaluation.

Ultimately, the study seeks to shed light on the implications of using AI-generated code in high-security domains and underscores the necessity of rigorous validation before deploying such technologies in real-world financial applications.

3 Problem

The increasing adoption of AI-assisted development tools still fails to maintain a systematic validation of the security of AI-generated code. Current AI models are primarily optimized for generating functionally correct code rather than ensuring compliance with secure coding practices and security standards. As a result, an application may appear complete and usable while still containing implementation-level flaws [2].

The problem becomes more serious when AI-generated applications create a false sense of security. In many cases, generated code may appear correct because the main features work under normal conditions. However, functional correctness does not guarantee secure behavior. For example, a banking application may successfully perform login, deposit and withdrawal operations but still fail to handle abnormal inputs, malicious requests, authentication failures or extreme cases securely. This is particularly dangerous because banking systems rely on strict validation and secure authentication to protect the confidentiality, integrity and availability of the system.

This raises major concerns when AI is used to develop software for critical industries such as the banking sector, healthcare, aviation and others. When implementing AI-generated codes, it may contain hidden vulnerabilities including insecure authentication mechanisms, improper use of cryptographic functions and insufficient input validation. These issues are difficult to detect because the generated code often appears correct and complete from a functional perspective.

Furthermore, recent studies highlight that trusting AI-generated code strongly requires an individual's level of expertise. Experienced developers are able to identify overly complex or potentially unsafe code and apply necessary corrections. Less experienced users may lack the ability to do so. This creates a gap in verification when the individual responsible for developing the application does not fully understand the generated code.

The term vibe coding which is first coined by Karpathy in 2025 is relatively new [3]. As a result, there is limited empirical analysis evaluating whether AI-generated code can reliably implement security-critical components in real-world applications. This gap makes it challenging to determine the safety of deploying such code in environments that require strong guarantees of confidentiality, integrity, and availability.

Therefore, a structured approach that combines static analysis with dynamic testing is required to effectively identify both implementation-level and logic-level vulnerabilities in AI-generated systems [4]. This approach enables a better understanding of the associated risks particularly in the context of banking applications.

4 Approach

The large language model (LLM) used in this study is ChatGPT Plus. It provides access to a more capable model variant as compared to the basic plan. The model is selected due to its stronger performance in structured code generation and multi-step reasoning which are critical for producing consistent and secure application features. This model was employed to generate and refine the backend implementation of a banking application including authentication logic, transaction handling and input validation mechanisms. The inital prompt is as follows:

"Create a simple yet functional secured banking application that includes features like user registration, login, deposit and withdraw as well as AES encryption for sensitive data."

Additional prompts were also provided to refine and complete the application. This is because the initial response did not fully generate all required functionalities in a single attempt. These follow-up prompts were used to ensure that the system functions as intended and covers core banking features. All codes generated by ChatGPT was directly used without manual modification and implemented using Visual Studio Code. Since this research focuses on the code security, no user interface was developed. Hence, requests were tested through Postman.

After ensuring that the application functions correctly, security analysis was conducted using a combination of static and dynamic techniques. Static analysis was performed using Semgrep to identify known vulnerability patterns while additional manual review was conducted with reference to Common Weakness Enumeration (CWE) categories. CWE is a community-developed taxonomy of software and hardware security vulnerabilities that provides a standardized identification and classification system for architectural, design, and code-level flaws [5]. By utilizing CWE, the analysis ensures that identified risks are mapped to industry-recognized security weaknesses, facilitating more effective remediation strategies.

Dynamic testing was carried out using Postman to analyze the runtime behavior of the application particularly for detecting vulnerabilities at logic level that may be missed by the static analysis. The testing process involved sending HTTP requests to various API endpoints including registration, login, deposit and withdrawal functions. An observation of the system's responses is carried out for both normal and abnormal conditions. To ensure a systematic evaluation, Boundary Value Analysis (BVA) technique was applied in designing the test cases. BVA was selected because prior empirical studies have shown that boundary-focused testing is highly effective in detecting software faults particularly those related to invalid ranges and input constraints [6]. This makes BVA suitable for evaluating banking applications where transaction values, authentication inputs and validation constraints are security-critical and highly sensitive to boundary conditions.Inputs were selected around critical boundary conditions including negative values, zero, maximum values and in-

valid data types. These test cases were used to assess whether the application properly implements input validation rules and constraints.

5 Evaluation

5.1 Static Analysis

Semgrep reported four blocking findings after scanning the AI-generated banking application. The results indicate several security weaknesses related to missing protections, hardcoded credentials, and improper cryptographic implementation

Two findings correspond to the reuse of a hardcoded JWT secret across multiple operations. This indicate a repetition of insecure implementation rather than distinct vulnerabilities.

5.1.1 Missing CSRF Protection (CWE-352)

```
const express = require('express');
const app = express();
```

(CWE-352: Cross-Site Request Forgery CSRF) No CSRF protection middleware (e.g, csurf) is implemented. This makes the application vulnerable to CSRF attacks if deplyoed in a browser-based environment where authenticated requests can be forged. [7]

5.1.2 Hardcoded JWT Secret (CWE-798)

```
const SECRET = "mysecretkey";

const token = jwt.sign(
    { id: user.id }, SECRET);
const decoded = jwt.verify(
    token, SECRET);
```

(CWE-798: Use of Hard-coded Credentials) The JWT secret is hardcoded in the source code. If the source code is exposed, attackers can generate valid tokens and bypass authentication mechanisms. [8]

5.1.3 AES-GCM Missing Tag Length (CWE-347)

```
const decipher =
    crypto.createDecipheriv(
        algorithm,
        key,
        Buffer.from(parsed.iv, 'hex')
    );
```

(CWE-347: Improper Verification of Cryptographic Signature) The AES-GCM implementation does not explicitly validate the authentication tag length. This may allow attackers to manipulate ciphertext and bypass integrity checks.

The findings indicate that the static analysis tools are effective in detecting vulnerabilities at the implementation level such as hardcoded credentials and missing security mechanisms. [9]

5.2 Dynamic Analysis

Dynamic testing was performed using Postman based on Boundary Value Analysis (BVA). Test cases were designed using boundary inputs such as negative values, zero, maximum values and invalid data types to evaluate whether the application properly enforces input validation and transaction constraints.

5.2.1 Missing Input Validation (CWE-20)

```
app.post('/deposit', auth, (req, res)=>
{
    const { amount } = req.body;

    const newBalance = account.balance
    + amount;
});
```

Input	Expected	Actual
-100	Reject	Accepted
0	Reject	Accepted
”abc”	Reject	Accepted
abc	Reject	Crash
999999999	Limit or reject	Accepted

Table 1: Manual Testing Results for Node.js API

(CWE-20: Improper Input Validation) When the above test cases were sent, the system accepts all numerical values. When non-numerical value is sent, the system fails to throw an exception and catch the failure leading to crash. String test case is also tested and the system accept the input and append it to the value. The application does not validate a user input before processing transactions. This allows invalid or malicious values to be accepted and exposed to the risk of incorrect system. [10]

5.2.2 Boundary Check Failure (CWE-1284)

```
const newBalance = account.balance +
                   amount;
```

(CWE-1284: Improper Validation of Specified Quantity) Negative deposit reduces the balance instead of being rejected. The application fails to enforce boundary constraints on transaction values. Negative values should not be accepted for deposit operations. [11]

5.2.3 Missing Token Expiration (CWE-613)

```
const token = jwt.sign({
id: user.id }, SECRET
);
```

(CWE-613: Insufficient Session Expiration) The authentication token does not have an expiration time which allows it to remain valid forever. This increases the risk of token compromise. Attackers may reuse the token to gain unauthorized access to the system and perform actions on behalf of the legitimate user. [12]

5.2.4 Weak Authentication Error Handling (CWE-204)

```
if (!user) return res.status(400).
           send('User not found');
if (!match) return res.status(400).
           send('Wrong password');
```

(CWE-204: Observable Response Discrepancy) Attackers can detect valid and invalid usernames based on different error responses. This enables user enumeration attack which then allows attackers to identify registered accounts and target them for brute-force attacks. [13]

5.2.5 No Input Validation (CWE-20

```
const { username, password } = req.body;
```

(CWE-20: Improper Input Validation) The application does not perform validation on user input such as enforcing non-empty username or password, length constraints or format requirements. This may result in weak credentials or malformed inputs being accepted. [10]

Overall, the evaluation demonstrates that the AI-generated banking application contains multiple security weaknesses despite appearing functionally correct during normal operation. The combination of static analysis and dynamic testing successfully identified both implementation and logic level vulnerabilities including insecure authentication handling, improper cryptographic usage and insufficient input validation. These findings suggest that AI-generated code should not be assumed secure solely based on functional behavior and must undergo systematic security verification before deployment in security-sensitive environments.

6 Related Work

Recent advancements in large language models have introduced new forms of AI-assisted software development such as developers using natural language to describe desired functionality and receive executable

code in return. This practice has evolved beyond traditional autocomplete tools into what is now commonly referred to as vibe coding. According to Ray, (from the first research paper) vibe coding is defined as an intent-driven development paradigm in which natural-language prompts can be used to generate complete software artifacts including project structures, APIs, user interfaces, tests and deployment configurations. Unlike conventional AI coding assistants that mainly provide token-level suggestions, vibe coding systems support broader automation through prompt engineering, context management and agentic workflows.

Vibe coding has gained popularity because it emphasized speed and accessibility. According to Fawzy et al., it involves creating software by expressing goals in natural language and iteratively prompting AI code generation tools [14]. It often involves very little scrutinity or review of the output. Their review of grey literature highlights that users are drawn to vibe coding for its quick results, ease of use, creative potential, learning opportunities and inclusivity. This makes it particularly valuable for rapid prototyping thus enables people without any programming background and beginners to build functional applications without deep technical expertise. At the same time, the study points out a clear trade-off which is sacrificing verification, maintainability and rigorous quality assurance.

Although vibe coding improves productivity, previous studies consistently report concerns regarding the quality and security of codes generated by AI. Ray points to issues such as hallucinations, vulnerabilities, technical debt, declining developer skills and governance challenges such as key obstacles to adoption [4]. The review further emphasizes that automated quality assurance which includes static analysis, dynamic testing and security scanning should be integrated into development workflows rather than treated as an afterthought. This is important because AI-generated code can look functional on the surface but may still break secure coding standards, especially when prompts fail to specify security requirements.

Fawzy et al. further reveal that quality assurance in vibe coding tends to be weak and inconsistent [14]. Their study found that skipping QA was the most frequent approach, followed by manual testing or edits, blind trust in generated code, reliance on AI for QA and reprompting instead of debugging. This pattern is especially concerning in security-sensitive contexts since many users accept code once it runs without verifying how it handles malicious inputs, edge cases, authentication errors or cryptographic requirements. The researchers also introduce the idea of *vulnerable developers* which are individuals who can create applications with AI tools but lack the skills to debug, validate or secure the code they produce.

Mitchell and Shaaban also argue that vibe coding is vulnerable to technical debt and security issues because LLMs struggle to reconcile accumulated human-imposed constraints over long interactions [15]. As developers refine applications through repeated iterations, requirements can drift or conflict leading to inconsistent code. To mitigate this, they introduce "Vibe Reasoning" concept where a developer remains actively engaged in the verification loop. instead of relying solely on whether AI-generated code compiles or runs, Vibe Reasoning emphasizes autoformalizing specifications, validating outputs against defined requirements, and providing actionable feedback. This approach ensures that developers do not just accept functional-looking code at face value but systematically check it against correctness and security standards. In essence, it reframes vibe coding from a purely generative process into a collaborative one, where human oversight and structured verification safeguard against technical debt and security risks.

Existing literature therefore establishes that vibe coding offers clear benefits in productivity and accessibility but also introduces serious risks especially when the generated code is accepted without adequate review. Prior studies mainly focus on general AI-assisted development practices, user motivations, agentic risks or proposed verification frameworks. However, limited work has empirically examined the security of AI-generated implementations in a specific security-critical domain such as banking. Banking applications require strong authentication, secure session management, proper input validation and reliable cryptographic protection. In this context, vul-

nerabilities such as missing CSRF protection, hardcoded JWT secrets, weak AES-GCM configuration, missing token expiration and improper transaction validation can directly compromise user accounts and financial data.

To address this gap, this study evaluates a ChatGPT-generated banking application through both static and dynamic security analysis. Static analysis using Semgrep is applied to identify implementation-level weaknesses while dynamic testing using Postman and Boundary Value Analysis is used to observe runtime behavior under normal and abnormal inputs. The detected weaknesses are mapped to Common Weakness Enumeration categories to provide standardized vulnerability classification. By focusing on an AI-generated banking system, this research extends prior work on vibe coding risks and provides practical evidence of how AI-generated implementations may introduce security vulnerabilities in financial applications.

7 Conclusion

This research examined the security risks of AI-generated applications focusing on a prototype banking system generated using ChatGPT base model. The study was motivated by the increasing use of AI-assisted development tools and vibe coding practices where developers can rapidly create functional applications through natural-language prompts. Although this approach improves development speed and accessibility, the findings of this research show that functional correctness alone is not sufficient to determine whether an application is secure.

Through static analysis using Semgrep, the study identified multiple implementation-level vulnerabilities in the generated banking system. These included insecure authentication practices, hardcoded secrets, missing session expiration, insufficient input validation, improper cryptographic implementation and missing protection mechanisms such as CSRF prevention. These weaknesses demonstrate that AI-generated code may include security flaws even when the application appears complete and operational from a user's perspective.

Dynamic testing using Boundary Value Analysis further confirmed that the application did not securely handle several abnormal or invalid inputs. For example, transaction-related functions such as deposit and withdrawal operations required stricter validation to prevent negative values, invalid data types, unexpected inputs or boundary-related errors. This shows that the generated application was mainly able to support normal use cases but lacked sufficient defensive handling for edge cases and potentially malicious inputs.

The findings suggest that AI-generated code can create a false sense of security. Since the application can perform basic banking operations such as login, deposit and withdrawal, it may appear reliable on the surface. However, deeper analysis revealed vulnerabilities that could compromise the system if deployed in a real environment. This is particularly critical for banking applications.

Overall, this research highlights the importance of applying structured security evaluation to AI-generated software especially in security-critical domains. AI-assisted development tools should not be treated as code generators that is secured by default. Instead, the generated code should be reviewed, tested and validated using a combination of static and dynamic analysis before deployment. The use of CWE mapping also helps classify the identified issues according to recognized security weaknesses making the findings easier to understand and compare with existing security standards.

In conclusion, AI-assisted development provides clear benefits in terms of speed, convenience and accessibility. However, this study shows that generated code may still contain serious security weaknesses particularly when applied to banking systems. Therefore, AI-generated applications must undergo systematic security assessment before being trusted in real-world or security-sensitive environments.

References

[1] M. Waseem, A. Ahmad, K.-K. Kemell, J. Rasku, S. Lahti, K. Mäkelä, and P. Abrahamsson, "Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use," *arXiv preprint arXiv:2512.11922*, 2025.

[2] D. Gandhi, "Survey on systemic security risks in vibe coding and autonomous development workflows," *TechRxiv*, 2026.

[3] A. Karpathy, "Vibe coding," https://x.com/karpathy, 2025, post on X (formerly Twitter), accessed 2026.

[4] P. P. Ray, "A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions," *TechRxiv*, 2025.

[5] MITRE, "Common weakness enumeration (cwe)," https://cwe.mitre.org/, 2026, accessed: 2026-05-24.

[6] S. Reid, "An empirical analysis of equivalence partitioning, boundary value analysis and random testing," in *Proceedings Fourth International Software Metrics Symposium*, 1997, pp. 64–73.

[7] MITRE, "CWE-352: Cross-Site Request Forgery (CSRF)," https://cwe.mitre.org/data/definitions/352.html, 2026, accessed: 2026-06-02.

[8] ——, "CWE-798: Use of Hard-coded Credentials," https://cwe.mitre.org/data/definitions/798.html, 2026, accessed: 2026-06-02.

[9] ——, "CWE-347: Improper Verification of Cryptographic Signature," https://cwe.mitre.org/data/definitions/347.html, 2026, accessed: 2026-06-02.

[10] ——, "CWE-20: Improper Input Validation," https://cwe.mitre.org/data/definitions/20.html, 2026, accessed: 2026-06-02.

[11] ——, "CWE-1284: Improper Validation of Specified Quantity in Input," https://cwe.mitre.org/data/definitions/1284.html, 2026, accessed: 2026-06-02.

[12] ——, "CWE-613: Insufficient Session Expiration," https://cwe.mitre.org/data/definitions/613.html, 2026, accessed: 2026-06-02.

[13] ——, "CWE-204: Observable Response Discrepancy," https://cwe.mitre.org/data/definitions/204.html, 2026, accessed: 2026-06-02.

[14] A. Fawzy, A. Tahir, and K. Blincoe, "Vibe coding in practice: Motivations, challenges, and a future outlook – a grey literature review," *arXiv preprint arXiv:2510.00328*, 2025.

[15] J. Mitchell and Y. Shaaban, "Position: Vibe coding needs vibe reasoning: Improving vibe coding with formal verification," in *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages (LMPL '25)*, 2025.